Java on the Command Line

Sophomore Level: Projects, Libraries and Builds

Haluk O. Bingol

Faculty of Computer and Information Sciences
Yeditepe University

v2026-10-13

Outline

1. A Real Project
2. Compiler Options
3. The Class Path in Depth
4. Packaging
5. Testing and Documentation
6. Build Tools

A Real Project

A library and an application

Terminal codeoutput/Layout.txt

```
$ find . -type f | sort
./app/manifest.txt
./app/res/books.csv
./app/src/edu/yeditepe/books/Book.java
./app/src/edu/yeditepe/books/Catalog.java
./app/src/edu/yeditepe/books/Main.java
./build.bat
./build.sh
./util/src/edu/yeditepe/util/Table.java
```

- `util`: a small **library**, `edu.yeditepe.util.Table`, that prints aligned text tables
- `app`: the **application** `edu.yeditepe.books`; it reads a book list and prints it with `Table`
- `app/res`: **resources**, files the program reads at run time
- `build`: generated `.class` files; `dist`: what we ship
- `build.sh` / `build.bat`: the build as a script

Rule

Keep **sources** and **generated** files apart: you can delete `build` and `dist` at any time, and never put them in Git.

The library

codeoutput/Project/util/src/edu/yeditepe/util/Table.java — Java

```java
package edu.yeditepe.util;

import java.util.ArrayList;
import java.util.List;

/** A plain-text table with aligned columns. */
public class Table {
    private final List<String[]> rows = new ArrayList<>();

    /** Creates an empty table. */
    public Table() { }
    /** Adds a row; the first row is the header. */
    public Table add(String... cells) {
        rows.add(cells);
        return this;
    }

    /** Renders the table as text, one line per row. */
    public String render() {
        int[] w = new int[rows.get(0).length];
        for (String[] r : rows)
            for (int i = 0; i < w.length; i++) w[i] = Math.max(w[i], r[i].length());
        StringBuilder sb = new StringBuilder();
        for (String[] r : rows) {
            for (int i = 0; i < w.length; i++)
                sb.append(String.format("%-" + (w[i] + 2) + "s", r[i]));
            sb.append(System.lineSeparator());
        }
        return sb.toString();
    }
}
```

The application: `main` and the data

codeoutput/Project/app/src/edu/yeditepe/books/Book.java — Java

```java
package edu.yeditepe.books;

public record Book(String title, String author, int year) { }
```

codeoutput/Project/app/src/edu/yeditepe/books/Main.java — Java

```java
package edu.yeditepe.books;

import edu.yeditepe.util.Table;

public class Main {
    public static void main(String[] args) throws Exception {
        Table t = new Table().add("Title", "Author", "Year");
        for (Book b : Catalog.load()) {
            t.add(b.title(), b.author(), String.valueOf(b.year()));
        }
        System.out.print(t.render());
    }
}
```

codeoutput/Project/app/res/books.csv — CSV

```
Tutunamayanlar;Oğuz Atay;1972
İnce Memed;Yaşar Kemal;1955
Saatleri Ayarlama Enstitüsü;Ahmet Hamdi Tanpınar;1961
```

The application: reading a resource

codeoutput/Project/app/src/edu/yeditepe/books/Catalog.java — Java

```java
package edu.yeditepe.books;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.nio.charset.StandardCharsets;
import java.util.ArrayList;
import java.util.List;

public class Catalog {
    /** Reads books.csv from the class path (inside the JAR). */
    public static List<Book> load() throws IOException {
        List<Book> books = new ArrayList<>();
        var in = Catalog.class.getResourceAsStream("/books.csv");
        try (var r = new BufferedReader(new InputStreamReader(in, StandardCharsets.UTF_8))) {
            String line;
            while ((line = r.readLine()) != null) {
                String[] f = line.split(";");
                books.add(new Book(f[0], f[1], Integer.parseInt(f[2])));
            }
        }
        return books;
    }
}
```

`getResourceAsStream("/books.csv")` searches the **class path**, not the current folder, so the file is found in a folder *or* inside a JAR.

Building step by step

Terminal codeoutput/ByHand.txt

```
$ javac -d build/util util/src/edu/yeditepe/util/Table.java
$ mkdir -p dist/lib
$ jar --create --file dist/lib/table.jar -C build/util .
$ javac -d build/app -cp dist/lib/table.jar -sourcepath app/src app/src/edu/yeditepe/books/Main.java
$ find build -name "*.class" | sort
build/app/edu/yeditepe/books/Book.class
build/app/edu/yeditepe/books/Catalog.class
build/app/edu/yeditepe/books/Main.class
build/util/edu/yeditepe/util/Table.class
$ java -cp build/app:app/res:dist/lib/table.jar edu.yeditepe.books.Main
Title                       Author               Year
Tutunamayanlar              Oğuz Atay            1972
İnce Memed                  Yaşar Kemal          1955
Saatleri Ayarlama Enstitüsü Ahmet Hamdi Tanpınar 1961
```

`-sourcepath app/src`: `javac` finds `Catalog` and `Book` by itself. `-cp` names the library when compiling *and* when running; `app/res` is on the class path so `getResourceAsStream` finds `books.csv`. **Windows**: `-cp build\app;app\res;dist\lib\table.jar`.

Compiler Options

Warnings: -Xlint

codeoutput/Lint/Old.java **Java**

```java
import java.util.ArrayList;
import java.util.List;

public class Old {
    public static void main(String[] args) {
        List names = new ArrayList();       // raw type
        names.add("Ali");
        Integer n = new Integer(42);        // deprecated
        System.out.println(names + " " + n);
    }
}
```

By default `javac` only prints a short *Note*; `-Xlint:all` names every problem. Add `-Werror` to treat warnings as errors in a build.

Warnings: `-Xlint`

codeoutput/Lint/Old.java — **Java**

```java
import java.util.ArrayList;
import java.util.List;

public class Old {
    public static void main(String[] args) {
        List names = new ArrayList();       // raw type
        names.add("Ali");
        Integer n = new Integer(42);        // deprecated
        System.out.println(names + " " + n);
    }
}
```

Terminal — codeoutput/Lint.txt

```
$ javac Old.java
Old.java:8: warning: [removal] Integer(int) in Integer has been
        deprecated and marked for removal
        Integer n = new Integer(42);        // deprecated
                    ^
Note: Old.java uses unchecked or unsafe operations.
Note: Recompile with -Xlint:unchecked for details.
1 warning
$ javac -Xlint:all Old.java 2>&1 | grep warning
Old.java:6: warning: [rawtypes] found raw type: List
Old.java:6: warning: [rawtypes] found raw type: ArrayList
Old.java:7: warning: [unchecked] unchecked call to add(E) as a member of
         the raw type List
Old.java:8: warning: [removal] Integer(int) in Integer has been
        deprecated and marked for removal
4 warnings
```

By default `javac` only prints a short *Note*; `-Xlint:all` names every problem. Add `-Werror` to treat warnings as errors in a build.

Source file encoding: `-encoding`

Terminal codeoutput/Encoding.txt

```
$ file Turkce.java
Turkce.java: Java source, ISO-8859 text
$ javac Turkce.java 2>&1 | grep error
Turkce.java:3: error: unmappable character (0xE7) for encoding UTF-8
Turkce.java:3: error: unmappable character (0xE7) for encoding UTF-8
Turkce.java:3: error: unmappable character (0xFE) for encoding UTF-8
Turkce.java:3: error: unmappable character (0xFD) for encoding UTF-8
Turkce.java:3: error: unmappable character (0xF0) for encoding UTF-8
5 errors
$ javac -encoding ISO-8859-9 Turkce.java
$ java Turkce
çiçek, şık, dağ
$ iconv -f ISO-8859-9 -t UTF-8 Turkce.java > tmp.java
$ mv tmp.java Turkce.java
$ file Turkce.java
Turkce.java: Java source, Unicode text, UTF-8 text
$ javac Turkce.java
```

The file was saved in the old Turkish encoding ISO-8859-9. Since Java 18 `javac` assumes **UTF-8** on every system, so it rejects the bytes until we name the encoding, or convert the file (`iconv`; on **Windows** use the editor's *Save with encoding*). Older JDKs on Windows assumed Windows-1254 instead: the classic "works on my machine" bug. Save sources as UTF-8 and pass `-encoding UTF-8` in builds.

Compiling for an older Java: `-release`

codeoutput/Release/Modern.java **Java**

```java
public class Modern {
    public static void main(String[] args) {
        var line = "-".repeat(10);      // var: Java 10, repeat: Java 11
        System.out.println(line);
    }
}
```

`-release N` checks the language **and** the API of Java N and writes class files that Java N can run (version 52 = Java 8, 55 = 11, 61 = 17, 65 = 21, 69 = 25). Java 8 is *obsolete* as a target: `-Xlint:-options` hides that warning.

Compiling for an older Java: `-release`

codeoutput/Release/Modern.java — Java

```java
public class Modern {
    public static void main(String[] args) {
        var line = "-".repeat(10);      // var: Java 10, repeat: Java 11
        System.out.println(line);
    }
}
```

Terminal — codeoutput/Release.txt

```
$ javac --release 8 -Xlint:-options Modern.java 2>&1 | grep -E "error|symbol:"
Modern.java:3: error: cannot find symbol
  symbol:   class var
Modern.java:3: error: cannot find symbol
  symbol:   method repeat(int)
2 errors
$ javac --release 11 Modern.java
$ java Modern
----------
$ javap -v Modern.class | grep major
  major version: 55
$ javac --release 8 Hello.java 2>&1 | head -1
warning: [options] source value 8 is obsolete and will be removed in a future
         release
$ javap -v Hello.class | grep major
  major version: 52
```

`-release N` checks the language **and** the API of Java N and writes class files that Java N can run (version 52 = Java 8, 55 = 11, 61 = 17, 65 = 21, 69 = 25). Java 8 is *obsolete* as a target: `-Xlint:-options` hides that warning.

The Class Path in Depth

Order, wildcards and the CLASSPATH variable

codeoutput/ClassPath/App.java
Java

```java
public class App {
    public static void main(String[] args)
        {
        System.out.println("using version
        " + edu.Version.get());
    }
}
```

Two JARs with the same class: the **first** on the class path wins. `lib/*` means every JAR in `lib` (quote it on macOS; Windows: `-cp ".;lib\*"`). A `CLASSPATH` variable **replaces** the default `.`, so `App` itself disappears; prefer `-cp` and leave `CLASSPATH` unset.

Order, wildcards and the CLASSPATH variable

codeoutput/ClassPath/App.java
Java

```java
public class App {
    public static void main(String[] args)
        {
        System.out.println("using version
        " + edu.Version.get());
    }
}
```

Terminal codeoutput/ClassPath.txt

```
$ ls lib
version-1.0.jar
version-2.0.jar
$ javac -cp lib/version-1.0.jar App.java
$ java -cp .:lib/version-1.0.jar:lib/version-2.0.jar App
using version 1.0
$ java -cp .:lib/version-2.0.jar:lib/version-1.0.jar App
using version 2.0
$ java -cp ".:lib/*" App
using version 1.0
$ export CLASSPATH=lib/version-2.0.jar
$ java App
Error: Could not find or load main class App
Caused by: java.lang.ClassNotFoundException: App
$ java -cp .:$CLASSPATH App
using version 2.0
$ unset CLASSPATH
```

Two JARs with the same class: the **first** on the class path wins. `lib/*` means every JAR in `lib` (quote it on macOS; Windows: `-cp ".;lib\*"`). A `CLASSPATH` variable **replaces** the default `.`, so `App` itself disappears; prefer `-cp` and leave `CLASSPATH` unset.

Packaging

Executable JAR with a library: `Class-Path`

Terminal — `codeoutput/Manifest.txt`

```
$ sh build.sh
built: dist/books.jar
$ unzip -p dist/books.jar META-INF/MANIFEST.MF
Manifest-Version: 1.0
Main-Class: edu.yeditepe.books.Main
Class-Path: lib/table.jar
Created-By: 21.0.12.1 (Ubuntu)

$ cd dist
$ find . -type f | sort
./books.jar
./lib/table.jar
$ java -jar books.jar
Title                        Author               Year
Tutunamayanlar               Oğuz Atay            1972
İnce Memed                   Yaşar Kemal          1955
Saatleri Ayarlama Enstitüsü  Ahmet Hamdi Tanpınar 1961
$ cd ..
$ mkdir alone
$ cp dist/books.jar alone/
$ java -jar alone/books.jar 2>&1 | head -2
Exception in thread "main" java.lang.NoClassDefFoundError: edu/yeditepe/util/Table
        at edu.yeditepe.books.Main.main(Main.java:7)
```

`Class-Path: lib/table.jar` is relative to the **JAR's folder**, not the current folder: ship `books.jar` with `lib/`; alone it fails. `manifest.txt` must end with a newline, or its last line is silently ignored.

A “fat JAR: everything in one file

Terminal codeoutput/FatJar.txt

```
$ sh build.sh > /dev/null
$ mkdir fat
$ cd fat
$ jar xf ../dist/lib/table.jar
$ rm -r META-INF
$ cd ..
$ jar --create --file books-all.jar --main-class edu.yeditepe.books.Main -C build/app . -C app/res . -C fat .
$ jar tf books-all.jar | grep -v /$
META-INF/MANIFEST.MF
edu/yeditepe/books/Book.class
edu/yeditepe/books/Catalog.class
edu/yeditepe/books/Main.class
books.csv
edu/yeditepe/util/Table.class
$ java -jar books-all.jar
Title                       Author                Year
Tutunamayanlar              Oğuz Atay             1972
İnce Memed                  Yaşar Kemal           1955
Saatleri Ayarlama Enstitüsü Ahmet Hamdi Tanpınar  1961
$ ls -l dist/books.jar dist/lib/table.jar books-all.jar | awk '{print $5, $9}'
4777 books-all.jar
3516 dist/books.jar
1781 dist/lib/table.jar
```

One file is easy to ship, but you repackage other people’s code (check licences), files with the same name collide, and signed library JARs break. Build tools automate it (Maven Shade, Gradle Shadow).

Build scripts for macOS and Windows

codeoutput/Project/build.sh — bash

```
#!/bin/sh
# build.sh -- builds dist/ (macOS Terminal, Linux, Windows Git Bash)
set -e                                  # stop at the first error
rm -rf build dist
mkdir -p dist/lib
javac -d build/util util/src/edu/yeditepe/util/Table.java
jar --create --file dist/lib/table.jar -C build/util .
javac -d build/app -cp dist/lib/table.jar -sourcepath app/src \
      app/src/edu/yeditepe/books/Main.java
jar --create --file dist/books.jar --manifest app/manifest.txt \
    -C build/app . -C app/res .
echo "built: dist/books.jar"
```

codeoutput/Project/build.bat — command.com

```
@echo off
rem build.bat -- builds dist\ (Windows Command Prompt)
if exist build rmdir /s /q build
if exist dist rmdir /s /q dist
mkdir dist\lib
javac -d build\util util\src\edu\yeditepe\util\Table.java ||
        exit /b 1
jar --create --file dist\lib\table.jar -C build\util . || exit
        /b 1
javac -d build\app -cp dist\lib\table.jar -sourcepath app\src ^
      app\src\edu\yeditepe\books\Main.java || exit /b 1
jar --create --file dist\books.jar --manifest app\manifest.txt ^
    -C build\app . -C app\res . || exit /b 1
echo built: dist\books.jar
```

Run with `sh build.sh` (macOS Terminal, Git Bash) or `build.bat` (Windows Command Prompt). A script makes the build **repeatable**; a build tool (next section) adds dependencies, tests and conventions.

Testing and Documentation

Assertions are off by default: -ea

codeoutput/Assert/test/TableTest.java **Java**

```java
import edu.yeditepe.util.Table;

public class TableTest {
    public static void main(String[] args) {
        String out = new Table().add("a", "bb").add("ccc", "d").render();
        String[] lines = out.split(System.lineSeparator());
        assert lines.length == 2 : "two rows expected";
        assert lines[0].equals("a    bb  ") : "column 1 is 3 wide + 2 spaces";
        assert lines[1].endsWith("d") : "no padding after the last column";
        System.out.println("all tests passed");
    }
}
```

Without `-ea` (`-enableassertions`) every `assert` is skipped and the broken test "passes". The second assertion found a real bug: `render` pads the **last** column too. Real projects use JUnit.

Assertions are off by default: -ea

codeoutput/Assert/test/TableTest.java — Java

```java
import edu.yeditepe.util.Table;

public class TableTest {
    public static void main(String[] args) {
        String out = new Table().add("a", "bb").add("ccc", "d").render();
        String[] lines = out.split(System.lineSeparator());
        assert lines.length == 2 : "two rows expected";
        assert lines[0].equals("a    bb  ") : "column 1 is 3 wide + 2
        spaces";
        assert lines[1].endsWith("d") : "no padding after the last column"
        ;
        System.out.println("all tests passed");
    }
}
```

Terminal — codeoutput/Assert.txt

```
$ javac -d build/util util/src/edu/yeditepe/util/Table.
      java
$ javac -d build/test -cp build/util test/TableTest.java
$ java -cp build/util:build/test TableTest
all tests passed
$ java -ea -cp build/util:build/test TableTest
Exception in thread "main" java.lang.AssertionError: no
      padding after the last column
       at TableTest.main(TableTest.java:9)
```

Without `-ea` (`-enableassertions`) every `assert` is skipped and the broken test "passes". The second assertion found a real bug: `render` pads the **last** column too. Real projects use JUnit.

Documenting the library

Terminal codeoutput/Docs.txt

```
$ javadoc -quiet -d docs -sourcepath util/src edu.yeditepe.util
util/src/edu/yeditepe/util/Table.java:13: warning: no @param for cells
    public Table add(String... cells) {
                 ^
util/src/edu/yeditepe/util/Table.java:13: warning: no @return
    public Table add(String... cells) {
                 ^
util/src/edu/yeditepe/util/Table.java:19: warning: no @return
    public String render() {
                  ^
3 warnings
$ ls docs/edu/yeditepe/util
Table.html
package-summary.html
package-tree.html
```

`-sourcepath util/src edu.yeditepe.util` documents a whole package. The warnings list the missing tags: a method comment should describe each parameter (`@param`) and the result (`@return`). Ship `docs` with the library or publish it on a web site.

Build Tools

Build tools: Maven

```
<project>
  <modelVersion>4.0.0</modelVersion>
  <groupId>edu.yeditepe</groupId>
  <artifactId>books</artifactId>
  <version>1.0</version>
  <properties>
    <maven.compiler.release>21</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
  <dependencies>
    <dependency>
      <groupId>edu.yeditepe</groupId>
      <artifactId>table</artifactId>
      <version>1.0</version>
    </dependency>
  </dependencies>
</project>
```

`pom.xml` *describes* the project; Maven knows the steps:

Command	By hand
`mvn compile`	`javac -d ...`
`mvn test`	run the tests
`mvn package`	`jar -create ...`
`mvn clean`	`rm -rf target`

Sources go in `src/main/java`, resources in `src/main/resources`; libraries are downloaded from Maven Central. Same commands on Windows and macOS.

Not run here: Maven needs the internet, which these examples could not reach.

Build tools: Gradle

```
// build.gradle.kts
plugins {
    application
}

java {
    toolchain { languageVersion = JavaLanguageVersion
      .of(21) }
}

dependencies {
    implementation(files("lib/table.jar"))
}

application {
    mainClass = "edu.yeditepe.books.Main"
}
```

macOS	Windows
`./gradlew build`	`gradlew build`
`./gradlew run`	`gradlew run`
`./gradlew jar`	`gradlew jar`

`gradlew` (the *wrapper*) downloads the right Gradle version, so everyone builds the same way. The `application` plugin also makes start scripts for Windows and macOS.

Not run here (needs the internet). Either tool replaces `build.sh`; the concepts on these slides stay the same.

Summary

Recipes

Goal	Command (macOS; Windows: ; and \)
compile a whole project	`javac -d build -sourcepath src -encoding UTF-8 src/a/b/Main.java`
... against a library	`javac -d build -cp lib/x.jar -sourcepath src src/a/b/Main.java`
... for an older Java	`javac -release 17 ...`
with all warnings	`javac -Xlint:all -Werror ...`
library JAR	`jar -create -file lib/x.jar -C build .`
executable JAR + library	`jar -create -file app.jar -manifest m.txt -C build . -C res .`
run with libraries	`java -cp "build:res:lib/*" a.b.Main`
run tests with assertions	`java -ea -cp build:test Tests`
documentation	`javadoc -d docs -sourcepath src a.b`

Exercises

1. Add a column “Pages” to `books.csv` and print it; rebuild with `build.sh` or `build.bat`.
2. Fix the padding bug in `Table.render` so that `TableTest` passes with `-ea`.
3. Turn `TableTest` into a script step: the build must stop when a test fails (exit code).
4. Compile the project with `-release 17`. What changes? Can you go down to `-release 11`? (records!)
5. Ship the application as `dist/` (JAR + `lib/`) and as one fat JAR; compare sizes and startup.
6. Write the `pom.xml` for the project and build it with Maven on your own computer.

Questions?